Rafraîchir les données de développement avec anonymisation dans CloudNativePG

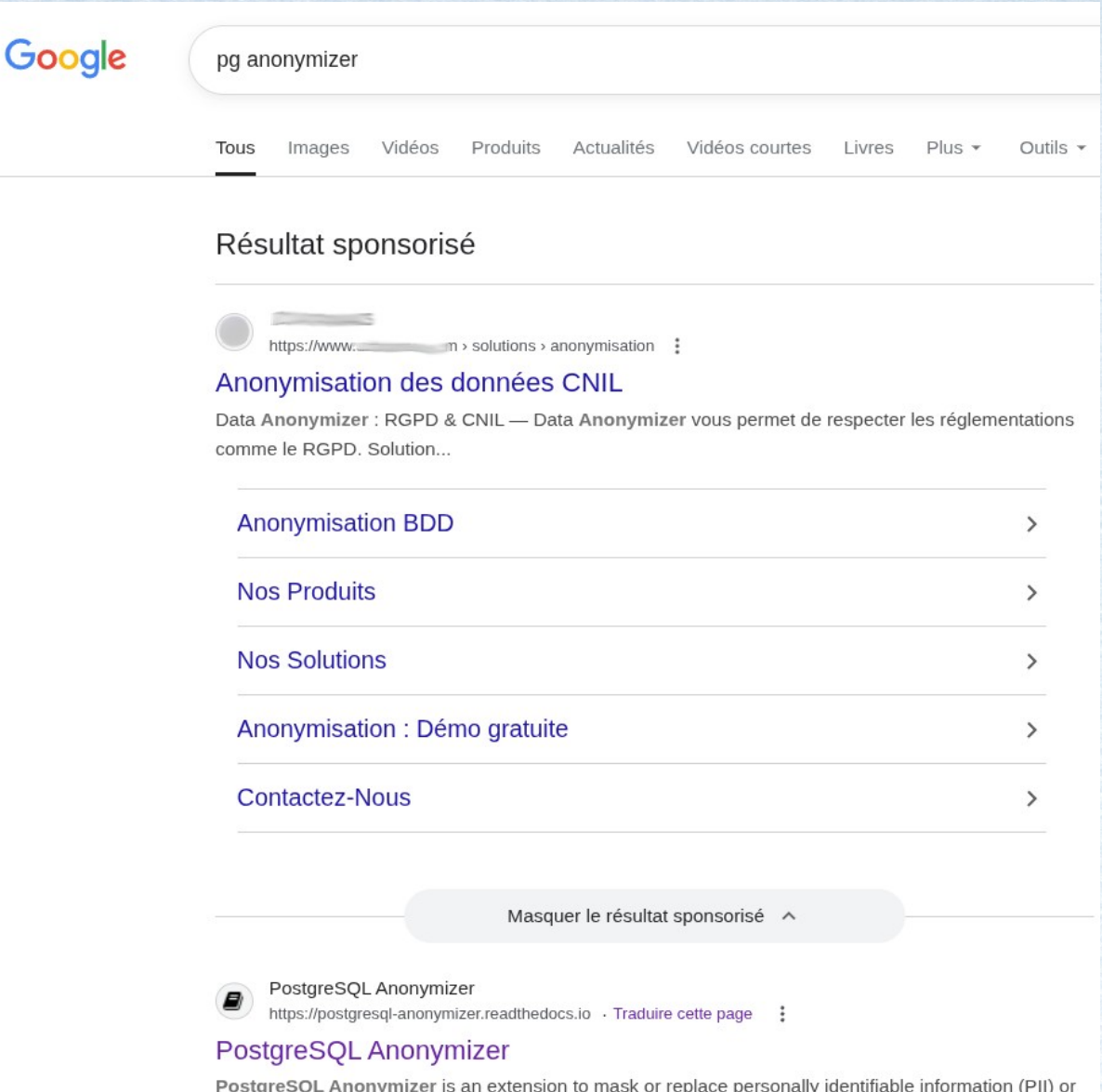
Atelier 14h – 16h

DALIBO
L'expertise PostgreSQL

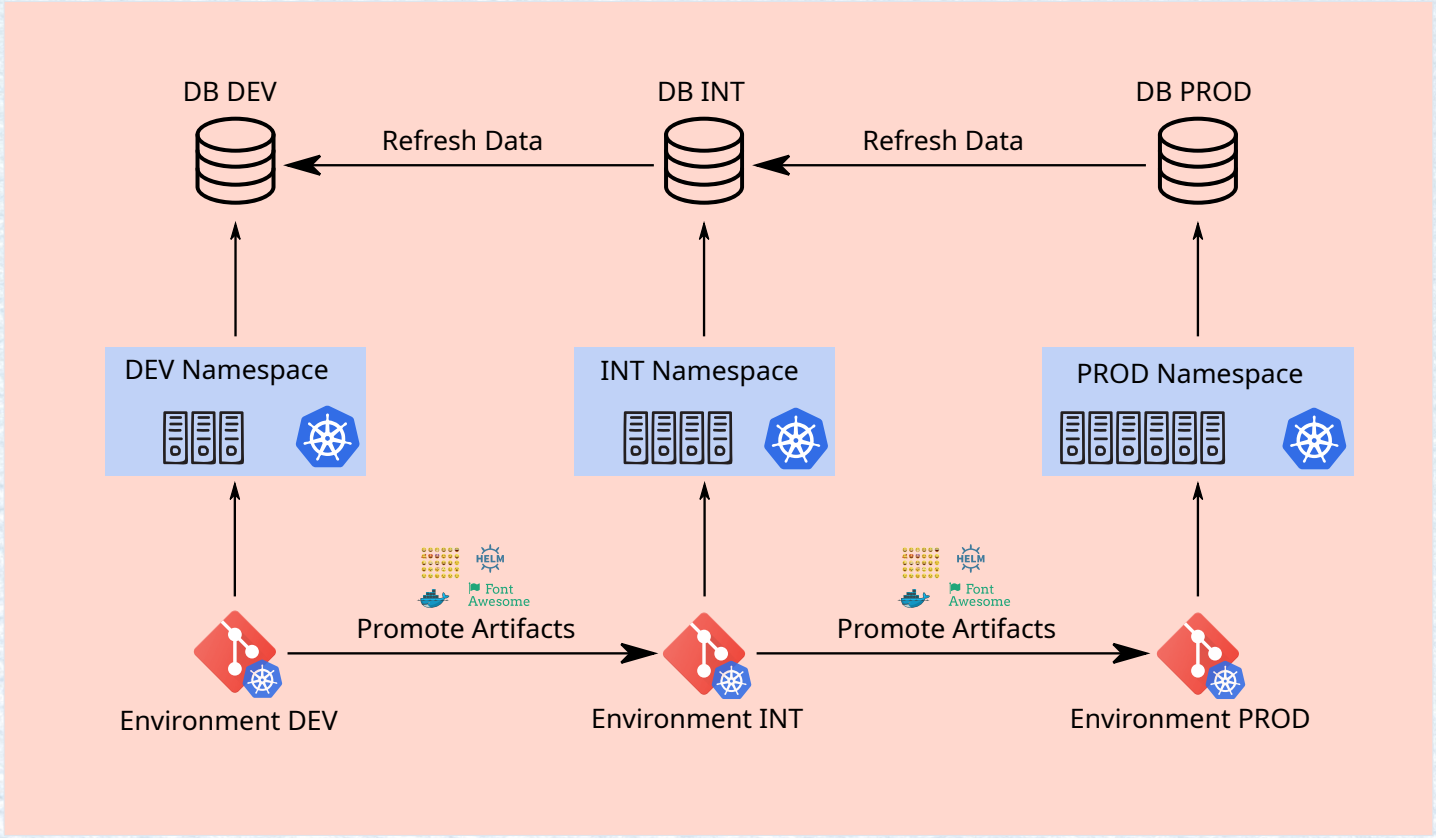
Alain Lesage – Dalibo
Julien Acroute – Camptocamp

Objectives

- Hands-on :
  - CloudNativePG
  - PostgreSQL Anonymizer
- Data Workflow
  - Keep data up-to-date
  - Data Anonymisation
  - Instant Delivery


Google
pg anonymizer
Tous
Images
Vidéos
Produits
Actualités
Vidéos courtes
Livres
Plus
Outils
Résultat sponsorisé
https://www.[redacted]m › solutions › anonymisation
Anonymisation des données CNIL
Data Anonymizer : RGPD & CNIL — Data Anonymizer vous permet de respecter les réglementations comme le RGPD. Solution...
Anonymisation BDD
Nos Produits
Nos Solutions
Anonymisation : Démo gratuite
Contactez-Nous
Masquer le résultat sponsorisé
PostgreSQL Anonymizer
https://postgresql-anonymizer.readthedocs.io · Traduire cette page
PostgreSQL Anonymizer
PostgreSQL Anonymizer is an extension to mask or replace personally identifiable information (PII) or

Data Workflow



PostgreSQL in Kubernetes

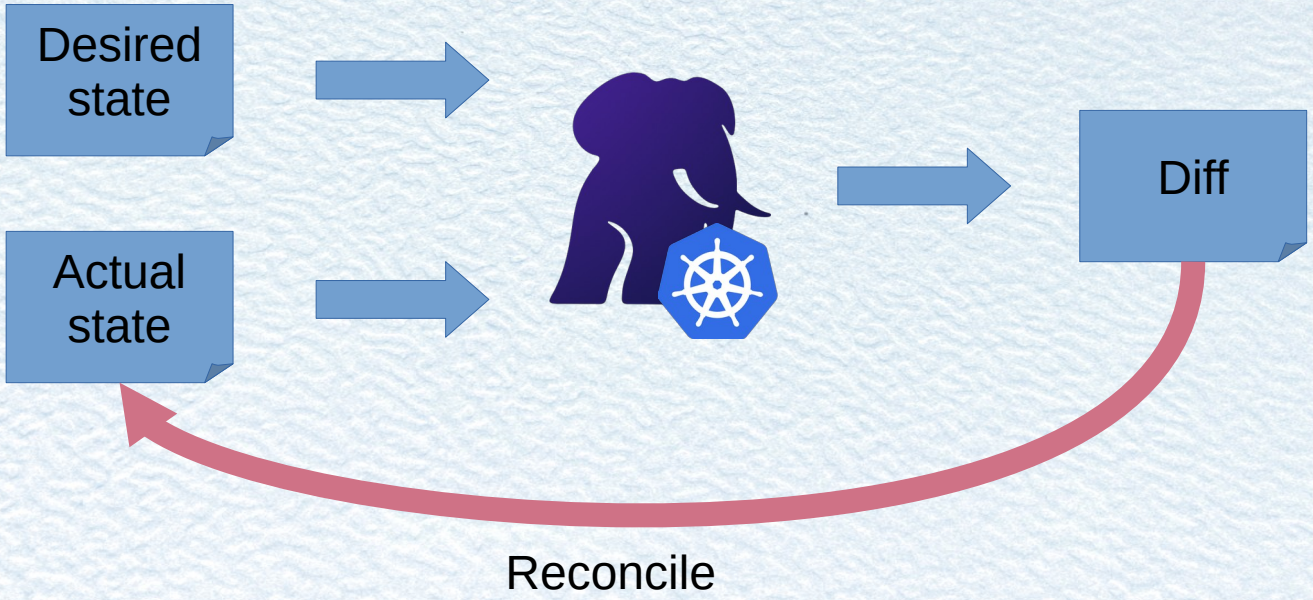
- Single Pod Deployment
- Kubernetes controller: Deployment, StatefullSet
- Patroni
- Kubernetes Operators
- CloudNativePG

CloudNative PG

- Kubernetes operator
- Manage PostgreSQL instances
- Full lifecycle
  - Init
  - Failover
  - Backup / Restore
- Automated Self-Healing

CloudNative PG

- Kubernetes operator



CPNG – Installation

Using a kubectl plugin from:

`https://github.com/cloudnative-pg/cloudnative-pg/releases/tag/v1.29.0`

```
#!/bin/bash
kubectl cnpg install generate | \
  kubectl apply --server-side --force-conflicts -f -
```

CloudNative PG

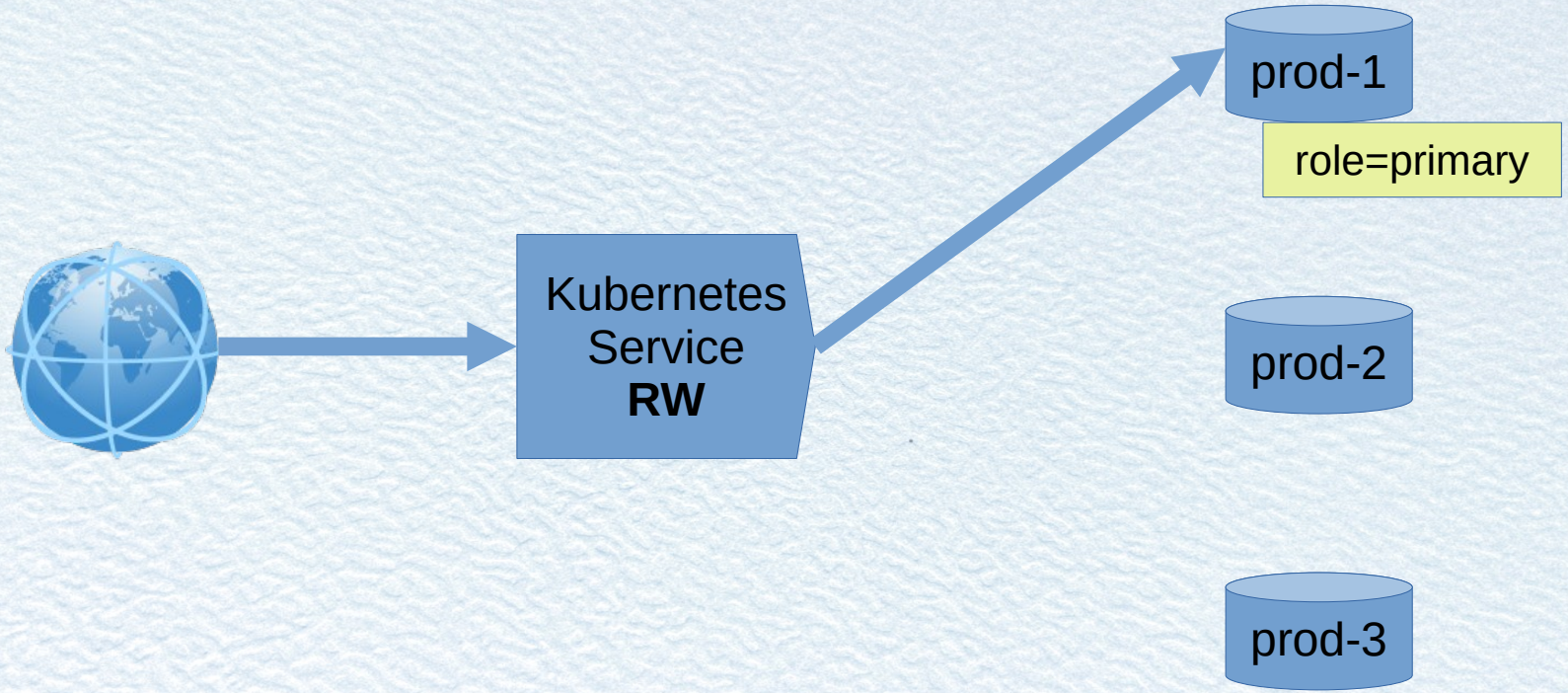
Offloads tasks to kubernetes

- Leader election, Split brain
- Single source of truth: `etcd`

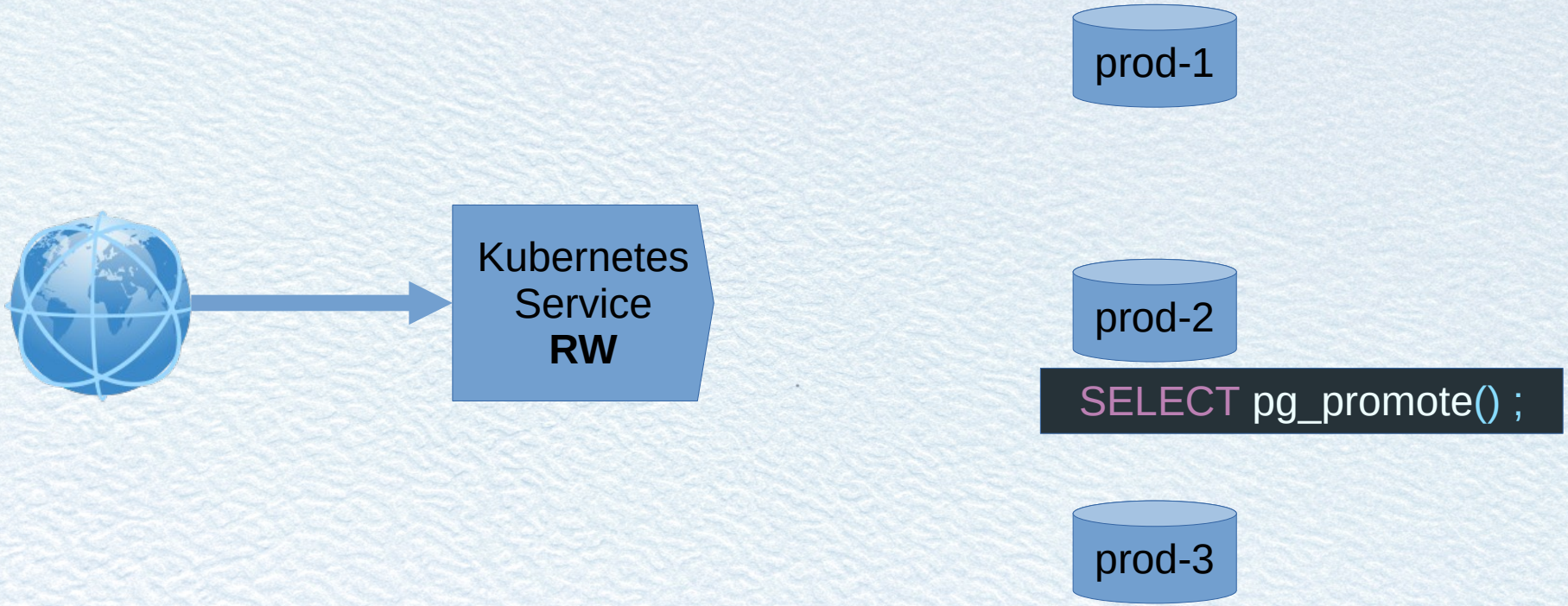
Integration with Kubernetes service

- Native load balancer
- Dynamic routing with labels: promotion

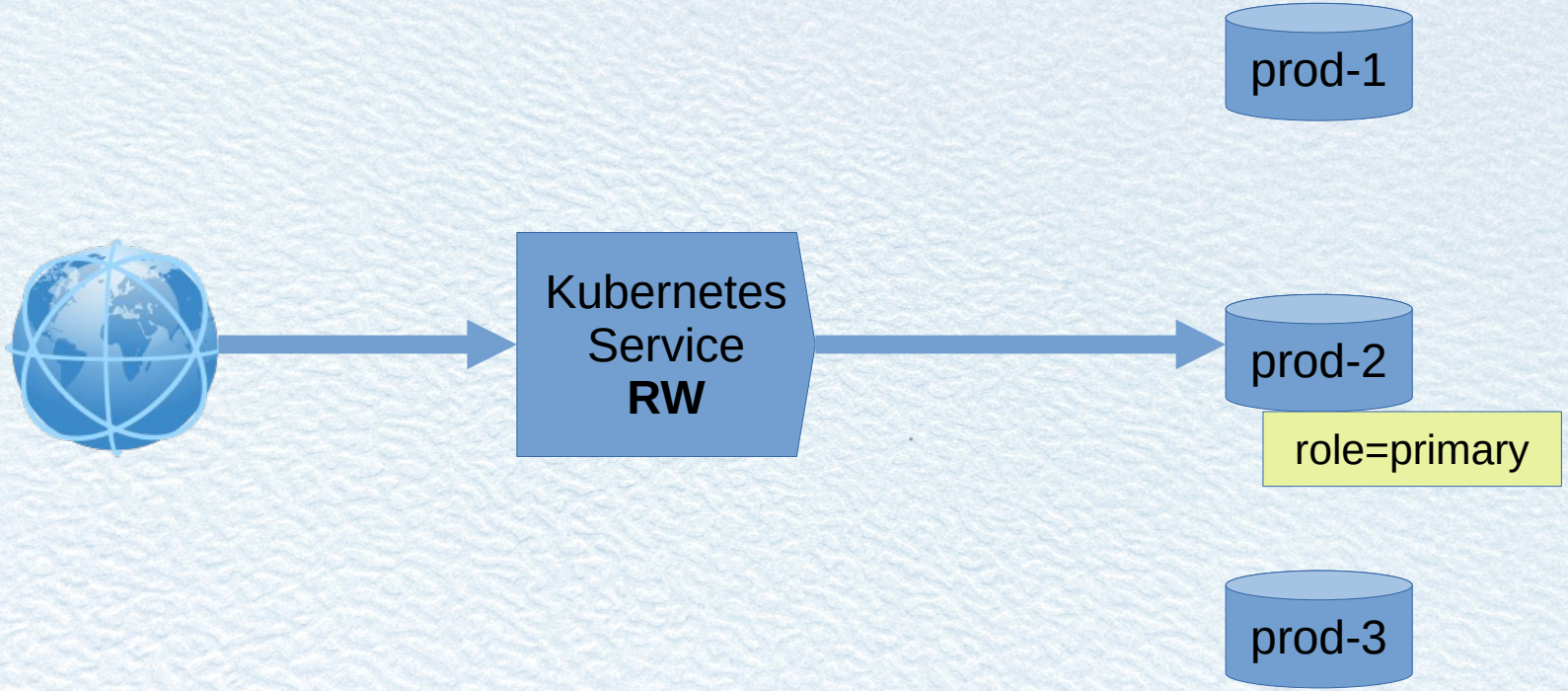
CloudNative PG



CloudNative PG



CloudNative PG

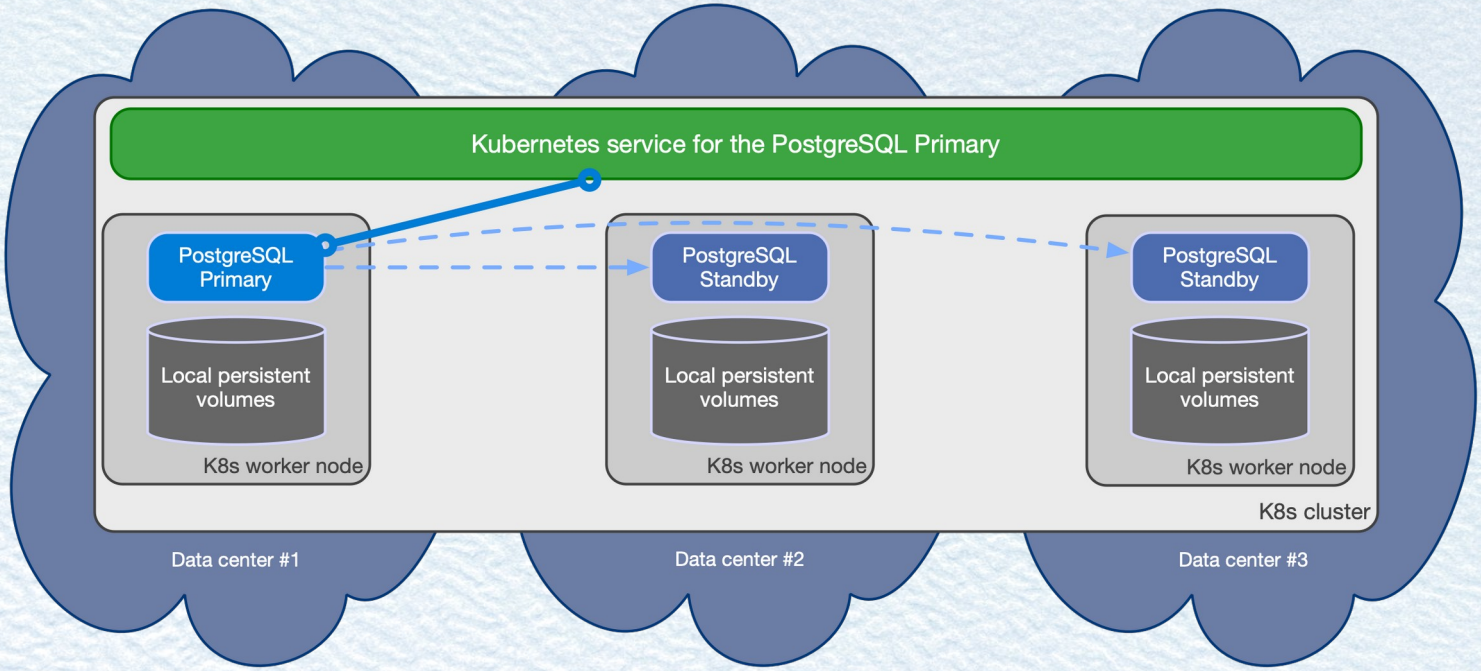


CloudNative PG

High Availability

- Kubernetes affinity / anti-affinity
- Ensure spreading of instances over:
  - Nodes
  - Datacenters

CloudNative PG



CloudNative PG

```yaml
---
apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: app1
  namespace: demo
spec:
  instances: 3
  storage:
    size: 10Gi
```

CloudNative PG

```
$ kubectl cnpg -n demo status app1
$ kubectl cnpg -n demo psql app1
$ kubectl cnpg -n demo pgbench app1 -- -i -s 10
$ kubectl cnpg -n demo pgbench app1 -- --time 30 --jobs 1
```

PostgreSQL Anonymizer

- Anonymization & Data Masking for Postgres
- Mask or replace personally identifiable information (PII)

PostgreSQL Anonymizer

```
CREATE TABLE people AS
    SELECT  153478       AS id,
            'Sarah'      AS firstname,
            'Conor'      AS lastname,
            '0609110911' AS phone;

CREATE EXTENSION anon;

SECURITY LABEL FOR anon ON COLUMN people.lastname
  IS 'MASKED WITH FUNCTION anon.dummy_last_name()';

SECURITY LABEL FOR anon ON COLUMN people.phone
  IS 'MASKED WITH FUNCTION anon.partial(phone,2,$$******$$,2)';
```

PostgreSQL Anonymizer

```
CREATE USER skynet;
SECURITY LABEL FOR anon ON ROLE skynet IS 'MASKED';

\connect - skynet
You are now connected to database "demo" as user "skynet"

SELECT * FROM people;
   id    | firstname | lastname  |   phone
---------+-----------+-----------+-------------
 153478 | Sarah     | Stranahan | 06******11
```

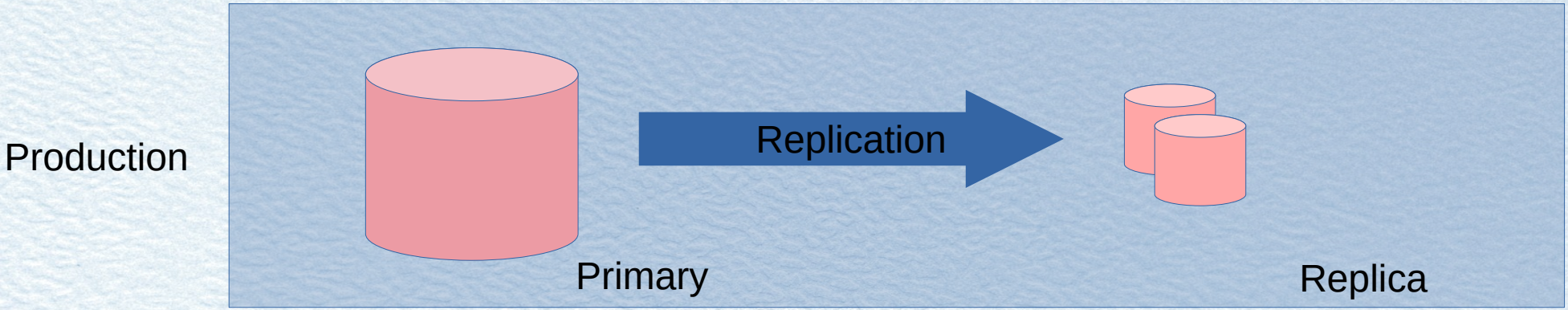
PostgreSQL Anonymizer Features

- Static masking
- Dynamic masking
- Anonymize Dump and replication
- Native PostgreSQL Rust Extension
- Anonymization roles : DBA / Dev
- Toolbox : generate test data

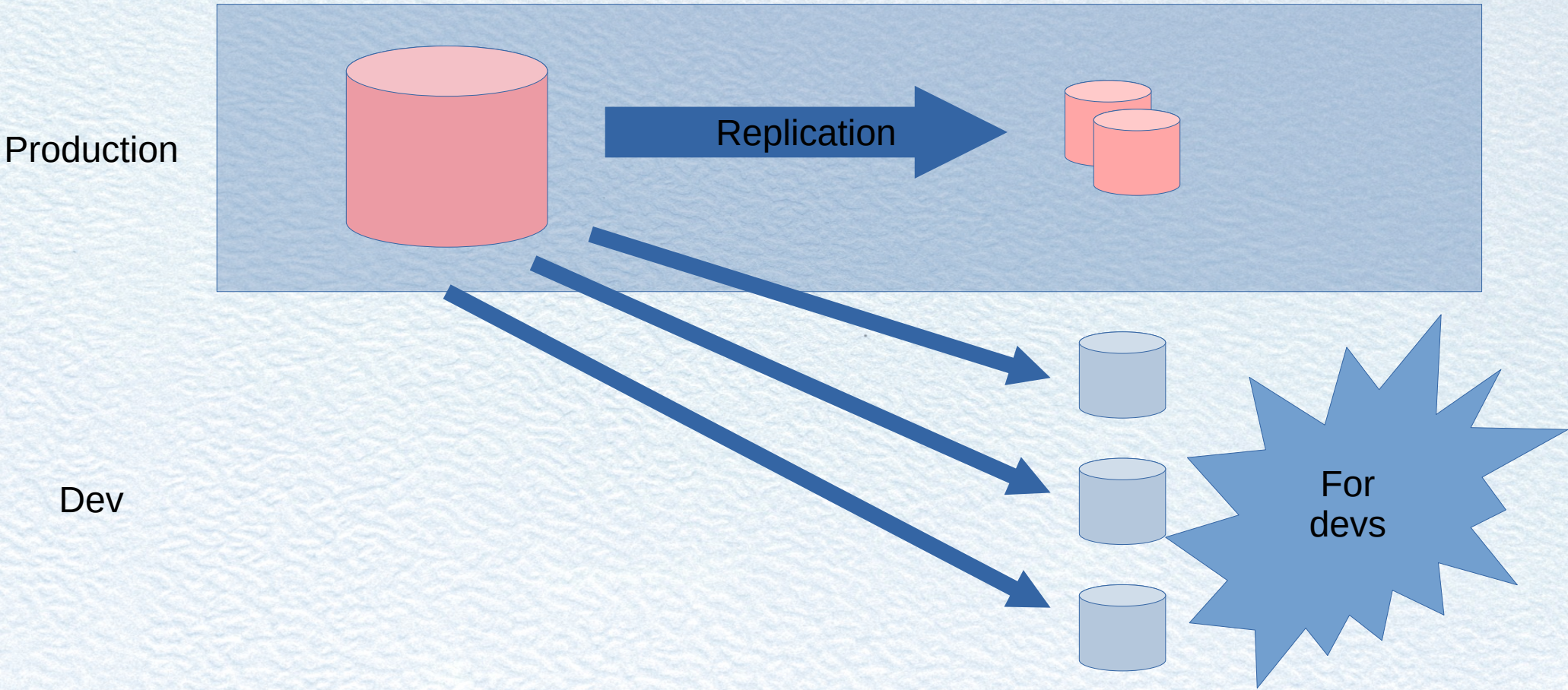
Data Workflow

- Objective : offer fresh data for development
  - A production cluster is managed by CNPG
  - A pool of replicas available for developpers

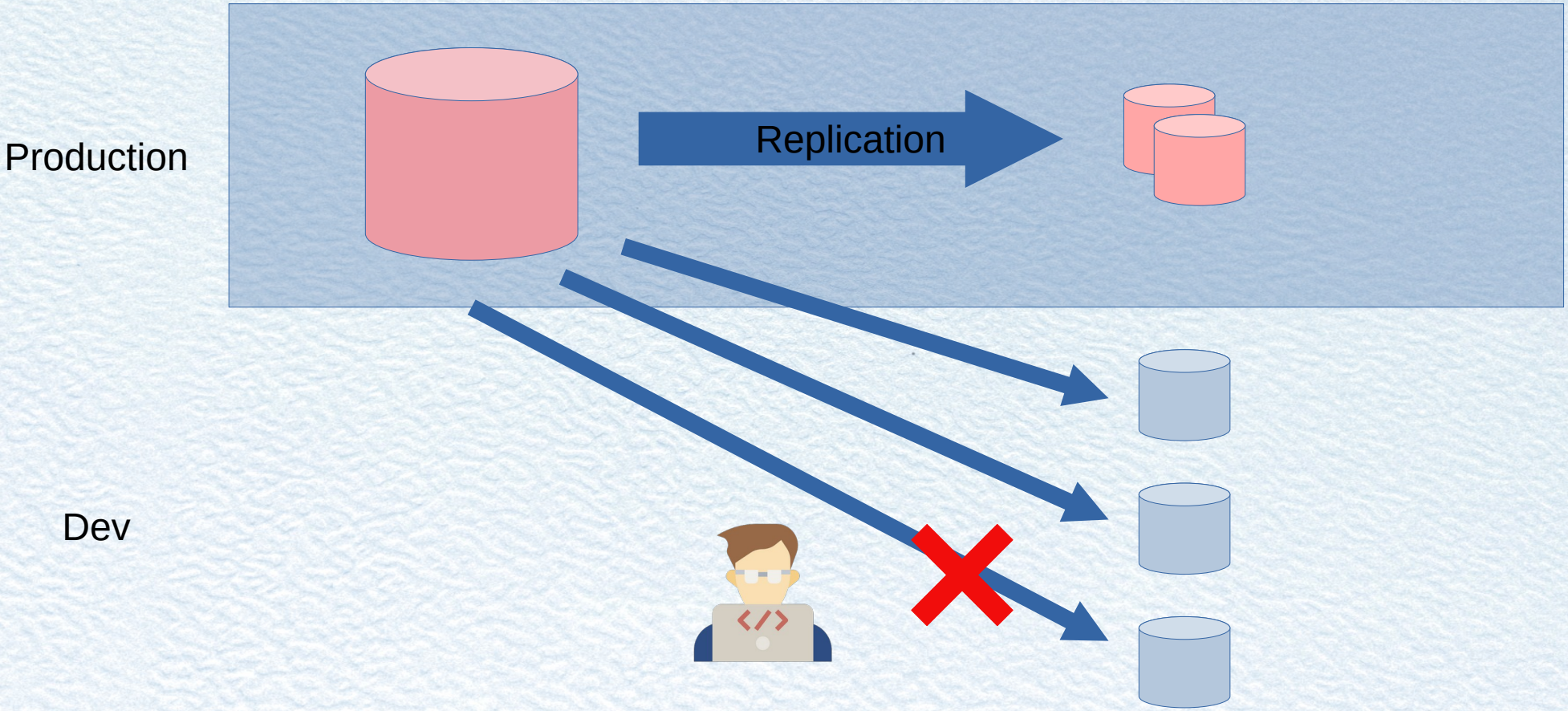
Data Workflow



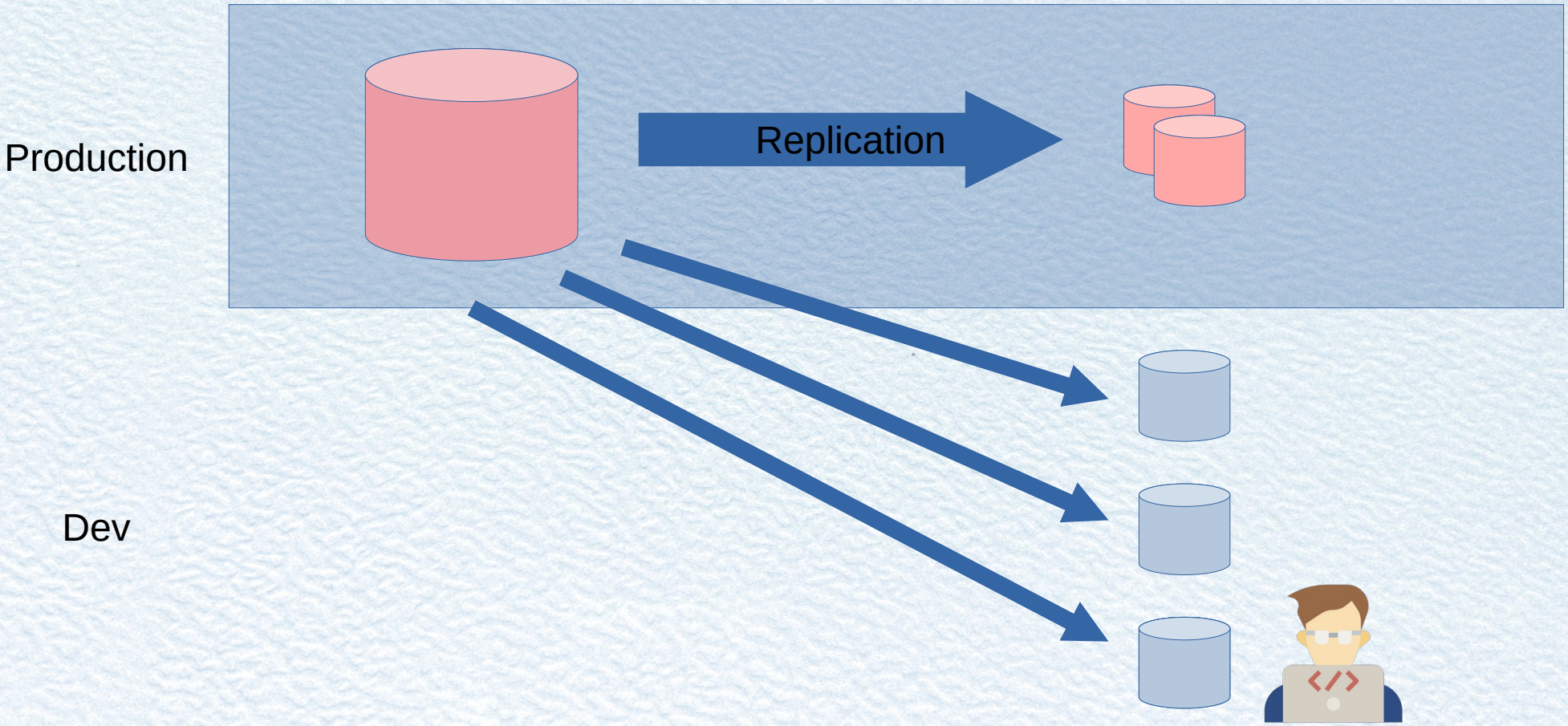
Data Workflow



Data Workflow



Data Workflow



Lab 0 – Install tools

Cards

- Install kubectl
- Download Kubeconfig
- Access Kubernetes cluster
- Install CloudNative PG plugin
- Access Instructions

Lab 1 – Deploy a Cluster

- Create a cluster « prod »
- Inspect Kubernetes objects
- Check cluster status
- Connect to the cluster
- Kill replica or primary

Lab 1 – Deploy a Cluster

- Create a cluster
  - 3 instances
  - 2 Gi
- Inspect Kubernetes objects
  - Pod, Service, Secret, Volume

```yaml
apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: prod
spec:
  instances: 3
  storage:
    size: 2Gi
```

```
kubectl get pod,service,secret,pvc -l "app.kubernetes.io/managed-by=cloudnative-pg"
```

Lab 1 – Deploy a Cluster

- Check cluster status
- Connect to the cluster
- Kill replica or primary

```
kubectl get cluster

kubectl cnpg status prod

kubectl cnpg psql prod
```

```
kubectl delete pod prod-3 # kill a replica

kubectl delete pod prod-1 # kill the primary
kubectl cnpg status prod  # notice timeline increment

kubectl delete pod \
  -l "app.kubernetes.io/managed-by=cloudnative-pg"
```

Lab 1 – Deploy a Cluster

- Create a cluster (3 instances, 2 Gi)
- Inspect Kubernetes objects
- Pod, Service, Secret, Volume
- Check cluster status
- Connect to the cluster
- Kill replica or primary

Follow instructions on
userX.campto.camp

Lab 2 – PostgreSQL Anonymizer

- Installation
- Inject Data
- Making rules
- Dynamic masking

Lab 2 – PostgreSQL Anonymizer

Installation

- Custom Image
- ClusterImageCatalog

```yaml
apiVersion: postgresql.cnpg.io/v1
kind: ClusterImageCatalog
metadata:
  name: camptocamp
spec:
  images:
    - major: 14
      image: ghcr.io/camptocamp/postgres:14
    - major: 15
      image: ghcr.io/camptocamp/postgres:15
    - major: 16
      image: ghcr.io/camptocamp/postgres:16
    - major: 17
      image: ghcr.io/camptocamp/postgres:17
    - major: 18
      image: ghcr.io/camptocamp/postgres:18
```

```yaml
apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: prod
spec:
  imageCatalogRef:
    apiGroup: postgresql.cnpg.io
    kind: ClusterImageCatalog
    name: camptocamp
    major: 18
  postgresGID: 999 # user id 26 is not define
  postgresUID: 999
```

Lab 2 – PostgreSQL Anonymizer

Installation on `postgres` database

```yaml
apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: prod
spec:
  ...
  bootstrap:
    initdb:
      postInitSQL:
      - |
        ALTER DATABASE postgres
          SET session_preload_libraries = 'anon';

        CREATE EXTENSION anon;

        ALTER DATABASE postgres
          SET anon.transparent_dynamic_masking TO true;
```

```yaml
apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: prod
spec:
  instances: 3
  imageCatalogRef:
    apiGroup: postgresql.cnpg.io
    kind: ClusterImageCatalog
    name: camptocamp
    major: 18
  postgresGID: 999 # user id 26 is not define
  postgresUID: 999
  postgresql:
    pg_hba:
      - host all all all trust
      - host replication all all trust
  storage:
    size: 2Gi
  bootstrap:
    initdb:
      postInitSQL:
        - |
          ALTER DATABASE postgres
            SET session_preload_libraries = 'anon';
          CREATE EXTENSION anon;
          ALTER DATABASE postgres
            SET anon.transparent_dynamic_masking TO true;
```

Lab 2 – PostgreSQL Anonymizer

Inject Data

```
CREATE TABLE people AS
    SELECT  153478       AS id,
            'Sarah'      AS firstname,
            'Conor'      AS lastname,
            '0609110911' AS phone;
```

Define masking rules

- lastname
- phone

```
SECURITY LABEL FOR anon ON COLUMN people.lastname
  IS 'MASKED WITH FUNCTION anon.dummy_last_name()';

SECURITY LABEL FOR anon ON COLUMN people.phone
  IS 'MASKED WITH FUNCTION anon.partial(phone,2,$$******$$,2)';
```

Dynamic masking

- « Masked » role

```
CREATE ROLE skynet LOGIN;

SECURITY LABEL FOR anon ON ROLE skynet IS 'MASKED';

GRANT pg_read_all_data to skynet;
```

Lab 2 – PostgreSQL Anonymizer

- Delete existing clusters
- Create a « prod » cluster with PostgreSQL Anonymizer
- Create a « people » table
- Define masking rules
- Create a masked user

Follow instructions on userX.campto.camp

Data Bootstrap

Multiple sources:

- SQL Queries
- `pg_dump` / `pg_dumpall`
- PITR Backup
- Streaming replication

Data Bootstrap

SQL Queries

- Inline SQL
- ConfigMap

```yaml
---
apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: init-sql
spec:
  instances: 3
  storage:
    size: 2Gi
  bootstrap:
    initdb:
      database: app
      owner: app
      postInitApplicationSQL:
        - |
          CREATE TABLE vegetables AS
              SELECT  1           AS id,
                      'broccoli' AS name;
          ALTER TABLE vegetables OWNER TO app;
```

Data Bootstrap

pg_dump/pg_dumpall

- `spec.externalClusters`
  - Declaration
  - No effect
- `spec.bootstrap.initdb.import`
  - Select database and roles

```yaml
apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: pg-dump
spec:
  instances: 1
  storage:
    size: 2Gi
  bootstrap:
    initdb:
      import:
        type: monolith
        databases:
          - app
        roles:
          - app
        source:
          externalCluster: upstream
  externalClusters:
    - name: upstream
      connectionParameters:
        host: init-sql-rw
        user: postgres
        dbname: app
```

Data Bootstrap

Replication

- Cluster wide import
- `spec.externalClusters`
- `spec.bootstrap.pg_basebackup`
- No streaming replication

```yaml
apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: pg-basebackup
spec:
  instances: 1
  storage:
    size: 2Gi
  bootstrap:
    pg_basebackup:
      source: upstream
  externalClusters:
    - name: upstream
      connectionParameters:
        host: init-sql-rw
        user: postgres
        dbname: app
```

Lab 3 – Data Bootstrap

Create a clone cluster:

- Define « prod » as external cluster
- Use masked role
- Set `spec.bootstrap.import`

Lab 3 – Data Bootstrap

Create a clone cluster:

- Define « prod » as external cluster
- Use masked role

```yaml
apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: prod-copy-green
spec:
  ...
  externalClusters:
    - name: prod
      connectionParameters:
        host: prod-rw
        user: skynet
        dbname: postgres
```

Lab 3 – Data Bootstrap

Create a clone cluster :

- Set `spec.bootstrap.import`

```yaml
apiVersion:
postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: prod-copy-green
spec:
  ...
  bootstrap:
    initdb:
      import:
        type: monolith
        databases:
          - postgres
        source:
          externalCluster: prod
        pgDumpExtraOptions:
          - --no-owner
          - --no-privileges
  externalClusters:
    - name: prod
      connectionParameters:
        host: prod-rw
        user: skynet
        dbname: postgres
```

```yaml
apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: prod-copy-green
spec:
  instances: 1
  storage:
    size: 2Gi
  imageCatalogRef:
    apiGroup: postgresql.cnpg.io
    kind: ClusterImageCatalog
    name: camptocamp
    major: 18
  postgresGID: 999
  postgresUID: 999
  bootstrap:
    initdb:
      import:
        type: monolith
        databases:
          - postgres
        source:
          externalCluster: prod
        pgDumpExtraOptions:
          - --no-owner
          - --no-privileges
  externalClusters:
    - name: prod
      connectionParameters:
        host: prod-rw
        user: skynet
        dbname: postgres
```

Lab 3 – Data Bootstrap

Create a clone cluster:

- Define « prod » as external cluster
- Use masked role
- Set `spec.bootstrap.import`

Replica Cluster

Replication between CNPG clusters

- Manage by `spec.replica`
- Use external clusters
- 2 modes :
  - Distributed Topology
  - Standalone Replica Clusters

Replica Cluster

Distributed Topology

- Across region
- Across kubernetes clusters
- Full lifecycle : Promote / Demote

Replica Cluster

Standalone Replica Clusters

- Failover cluster
- No controlled switchover
- One way promotion

Lab 4 – Replica Pool

Create a standalone replica cluster

- Keep `spec.externalClusters`
- Add a `spec.replica`
- Promote replica
- Apply static masking

Lab 4 – Replica Pool

- Reference prod cluster
- Replication credentials

```yaml
apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: prod-copy-blue
spec:
  ...
  externalClusters:
  - name: prod
    connectionParameters:
      host: prod-rw
      user: streaming_replica
      sslmode: verify-full
    sslKey:
      name: prod-replication
      key: tls.key
    sslCert:
      name: prod-replication
      key: tls.crt
    sslRootCert:
      name: prod-ca
      key: ca.crt
```

Lab 4 – Replica Pool

- Setup `spec.replica`
- Setup `spec.bootstrap`
- Streaming replication

```yaml
apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: prod-copy-blue
spec:
  ...
  replica:
    enabled: true
    source: prod
  bootstrap:
    pg_basebackup:
      source: prod
  externalClusters:
  - name: prod
    connectionParameters:
      host: prod-rw
      user: streaming_replica
      sslmode: verify-full
    sslKey:
      name: prod-replication
      key: tls.key
    sslCert:
      name: prod-replication
      key: tls.crt
    sslRootCert:
      name: prod-ca
      key: ca.crt
```

```yaml
apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: prod-copy-blue
spec:
  instances: 1
  storage:
    size: 2Gi
  imageCatalogRef:
    apiGroup: postgresql.cnpg.io
    kind: ClusterImageCatalog
    name: camptocamp
    major: 18
  postgresGID: 999
  postgresUID: 999
  replica:
    enabled: true
    source: prod
  bootstrap:
    pg_basebackup:
      source: prod
  externalClusters:
  - name: prod
    connectionParameters:
      host: prod-rw
      user: streaming_replica
      sslmode: verify-full
    SslKey: { name: prod-replication, key: tls.key }
    SslCert: { name: prod-replication, key: tls.crt }
    SslRootCert: { name: prod-ca, key: ca.crt }
```

Lab 4 – Replica Pool

Promotion

`spec.replica.enabled: false`

```
kubectl patch cluster prod-copy-blue \
  --type='merge' \
  -p '{"spec":{"replica":{"enabled":false}}}'
```

```
apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: prod-copy-blue
spec:
  ...
  replica:
    enabled: false
    source: prod
  bootstrap:
    pg_basebackup:
      source: prod
  externalClusters:
  - name: prod
    connectionParameters:
      host: prod-rw
      user: streaming_replica
      sslmode: verify-full
    sslKey:
      name: prod-replication
      key: tls.key
    sslCert:
      name: prod-replication
      key: tls.crt
    sslRootCert:
      name: prod-ca
      key: ca.crt
```

Lab 4 – Replica Pool

Static Masking

```
kubectl cnpg psql prod-copy-blue -- -c "SELECT anon.anonymize_database()"
```

Lab 4 – Replica Pool

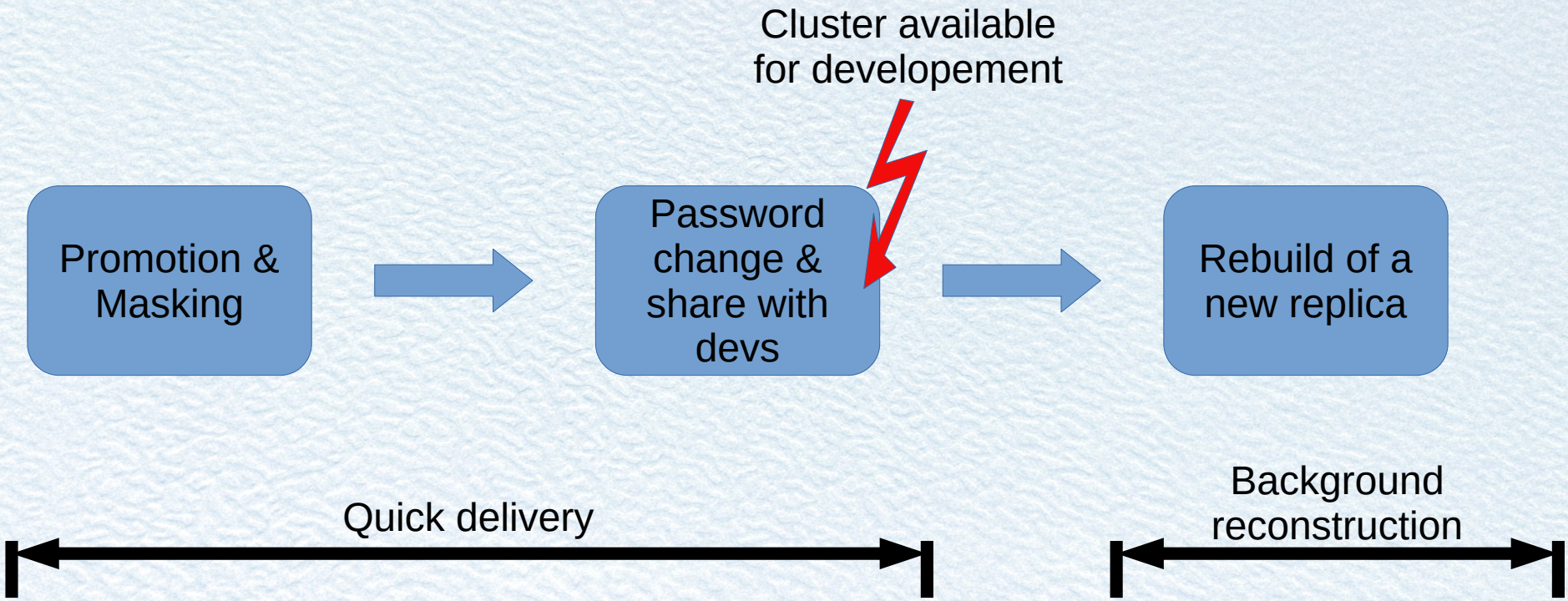
- New cluster « prod-copy-blue »
- Keep `spec.externalClusters`
- Set `spec.replica`
- Promote replica
- Apply static masking

Follow instructions on
userX.campto.camp

Automation

- Maintain a pool of replica
- Change password and share credentials
- Scripting :
  - Creation of a new replica
  - Promotion
  - Masking

Automation


Cluster available
for developement
Promotion &
Masking
Password
change &
share with
devs
Rebuild of a
new replica
Quick delivery
Background
reconstruction

Limitation

- Use replication credentials available in the namespace
- Static masking can be long on a big dataset
  - PostgreSQL Anonymizer v3 supports parallelization

Cloud Native App

- Application running in Kubernetes
- Using Kubernetes API
- User interface
- Manage lifecycle of clusters
  - Button to « pick » a replica
  - Button to « destroy » an old copy

Thanks

Contact: alain.lesage@dalibo.com

julien.acroute@camptocamp.com